



ماجراجویی با پایتون

سفری پروژه محور به دنیای برنامه نویسی

نویسنده: محمد مهدی قائمی

شناسنامه

سرشناسه
عنوان قراردادی
عنوان و نام پدیدآور
مشخصات نشر
مشخصات ظاهری
شابک
وضعیت
فهرست نویسی
یادداشت
موضوع
موضوع
رده بندی کنگره
رده بندی دیویی
شماره کتاب شناسی
ملی

ماجراجویی با پایتون

نویسنده: محمد مهدی قائمی

ناشر: انتشارات

شمارگان: نسخه

نوبت چاپ: اول، ۱۴۰۴

قیمت: تومان

شابک: ۹۷۸-۶۰۰- - -

سخن ناشر

پیش‌گفتار

به جایی که غیرممکن‌ها ممکن می‌شوند خوش آمدید!

تصور کن یک چوب جادویی داری که می‌توانی با آن، هر ایده‌ای که توی ذهنت می‌گذرد را به واقعیت تبدیل کنی. چه حس فوق‌العاده‌ای دارد، مگر نه؟ بشر همیشه عاشق این بوده که رویاهاش را بسازد و زندگیش را راحت‌تر کند؛ از روزی که اولین ابزارهای ساده را ساخت تا امروز که کامپیوترها دقیق‌ترین و سریع‌ترین دستیارهای ما هستند، هدف همیشه یک چیز بوده: زندگی راحت‌تر و ایده‌هایی بدون مرز.

حالا درست در نقطه‌ای ایستادی که می‌توانی پادشاه این دنیای دیجیتال باشی. برای صحبت با این دنیای شگفت‌انگیز و فرمان دادن به ماشین‌ها، به یک زبان مشترک نیاز داریم و چه انتخابی بهتر از پایتون؟

پایتون زبانی‌ست که هم‌زمان هم ساده‌ست و هم بی‌نهایت قدرتمند. یاد گرفتنش مثل آب خوردن است، اما نتیجه‌ای شگفت‌انگیز دارد. این همان کلید طلایی‌ست که دروازه‌های هوش مصنوعی، تحلیل داده‌ها و خلق نرم‌افزارهای بزرگ را به روی تو باز می‌کند. میلیون‌ها نفر در سراسر جهان با همین زبان، آینده‌ای روشن و هوشمند را رقم زدند و حالا نوبت درخشش توست.

در این کتاب، ما هم سفر یکدیگریم. قرار است در کنار هم، صفحه‌به‌صفحه لذت ببریم، با هم بازی بسازیم، اپلیکیشن خلق کنیم و پروژه‌هایی را اجرا نماییم که دیدن نتیجه‌شان حس غرور و شادی به تو می‌دهد. اینجا هر خط کد، یک قدم به سمت حرفه‌ای شدن و هر پروژه، پلی به سوی موفقیت توست.

آماده‌ای تا با هم این مسیر هیجان‌انگیز را شروع کنیم و آینده‌ای که دوست داری را با دست‌های خودت بسازی؟ جادوی واقعی درست از همین لحظه شروع می‌شود.

بزن بریم به سوی بی‌نهایت!

عهدنامه ماجراجویان (چه جوری این کتاب را بخونیم؟)

ماجراجوی عزیز، خواندن کتاب‌های برنامه‌نویسی مدل خاص خودش را دارد. برای اینکه بتوانی حداکثر استفاده را بکنی، پیشنهاد می‌شود از قانون سه مرحله‌ای پیروی کنی:

۱. **مرحله مشاهده:** ابتدا فصل را با دقت بخوان و مثال‌ها را نگاه و تحلیل کن. در این مرحله فقط داری با شکل‌وشمایل طلسم آشنا می‌شوی و یک ایده کلی از نحوه کارشان پیدا می‌کنی.
۲. **مرحله رونویسی:** وقتی به پروژه پایان فصل رسیدیم، کد نهایی را جلویت بگذار و خط به خط از روی آن تایپ کن.

- **نکته مهم:** کپی/پیست کردن ممنوع! جادوگر باید حرکت دست و املاي کلمات را حس کند. در حین نوشتن، سعی کن تحلیل کنی که هر خط دقیقاً چه کار می‌کند.

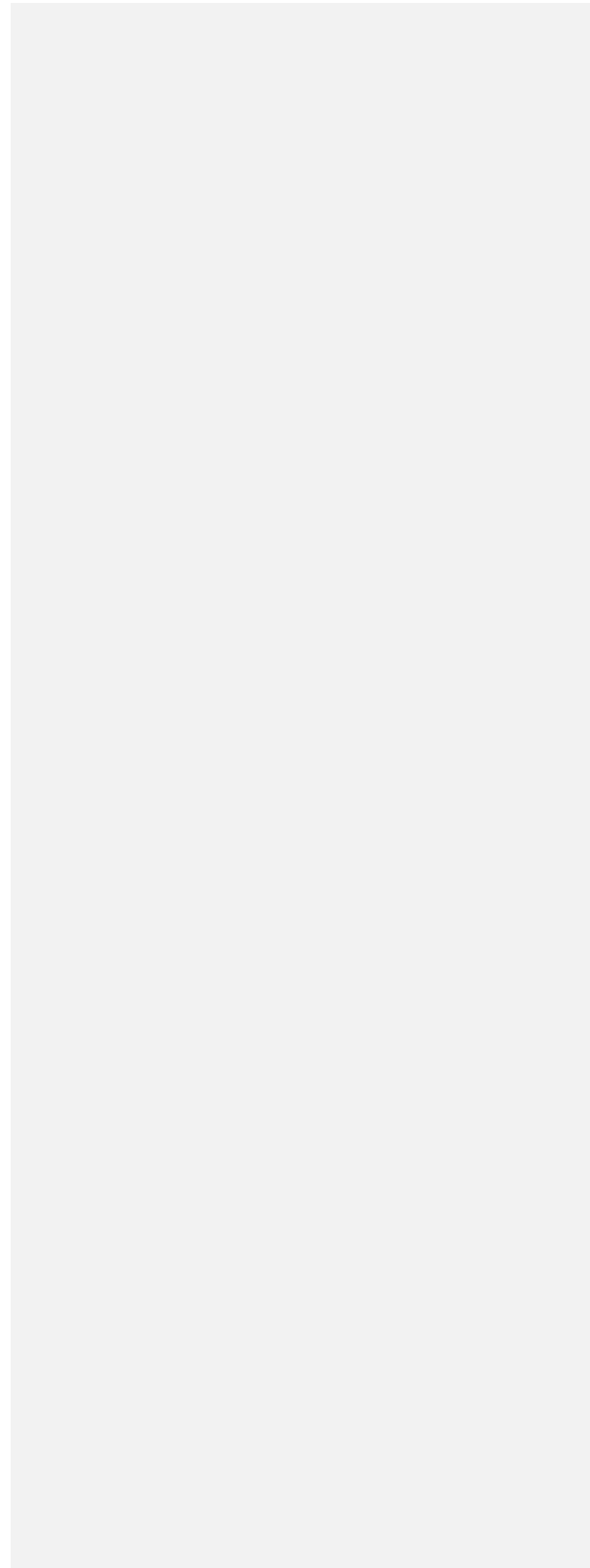
۳. **مرحله اجرای کور:** این مهم‌ترین مرحله است. حالا کتاب را ببند، سعی کن همان پروژه را از ذهن خودت و بدون نگاه کردن بنویسی.

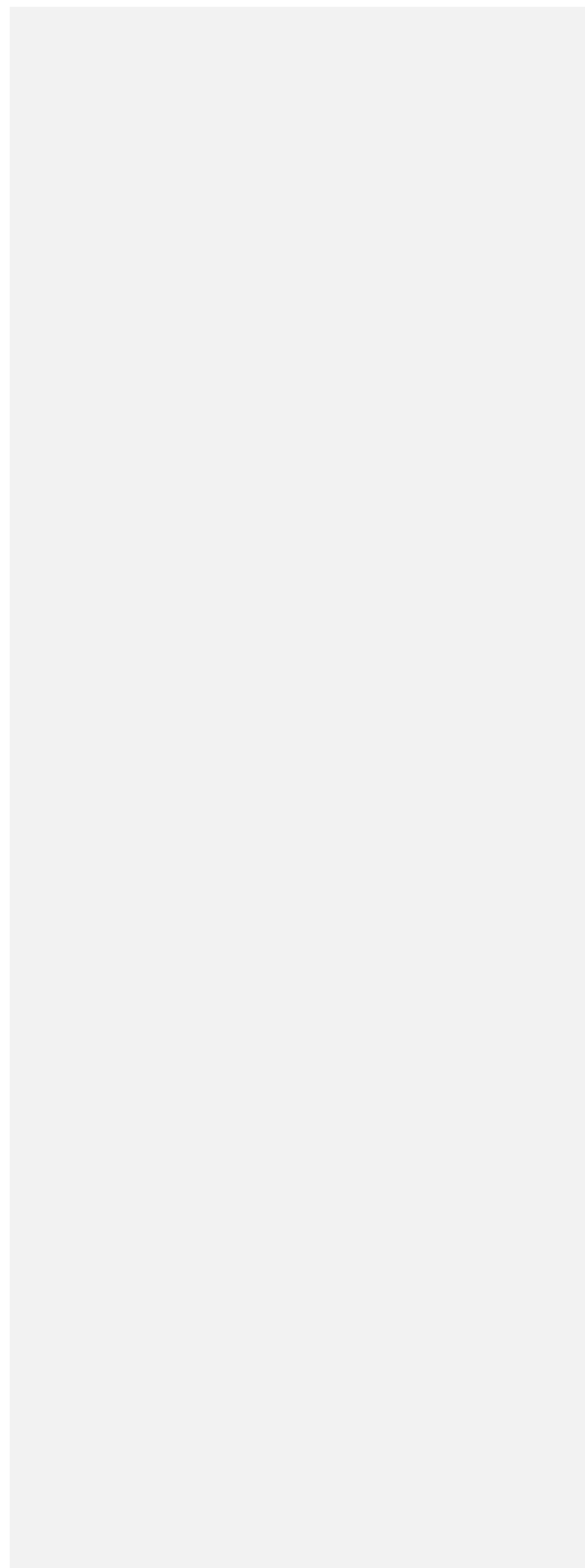
مأموریت‌های جانبی: در انتهای هر بخش، چالش‌هایی تحت عنوان مأموریت جانبی برای طراحی شده‌است. برخلاف پروژه‌های اصلی، برای این مأموریت‌ها باید سعی کنی خودت از ابتدا و بدون نگاه کردن به کد حلشان کنی و اگر موفق نشدی، جواب‌ها را بخوانی! تو باید با استفاده از دانشی که تا آن لحظه به دست آوردی و خلاقیت خودت، راه حل را پیدا کنی. اینجاست که یادگیری کامل می‌شود!

قانون طلایی: تا زمانی که نتوانستی پروژه پایان فصل را بدون نگاه کردن به کتاب بنویسی و مأموریت‌های جانبی را با موفقیت پشت سر بگذاری، یعنی هنوز طلسم‌ها را کاملاً یاد نگرفتی و نباید به فصل بعد بروی! (اینجا بیشتر تمرین کن، چون فصل‌های بعد این پیش‌فرض را دارند که تو مباحث فصل قبل را خوب یاد گرفته‌ای).

درباره نویسنده

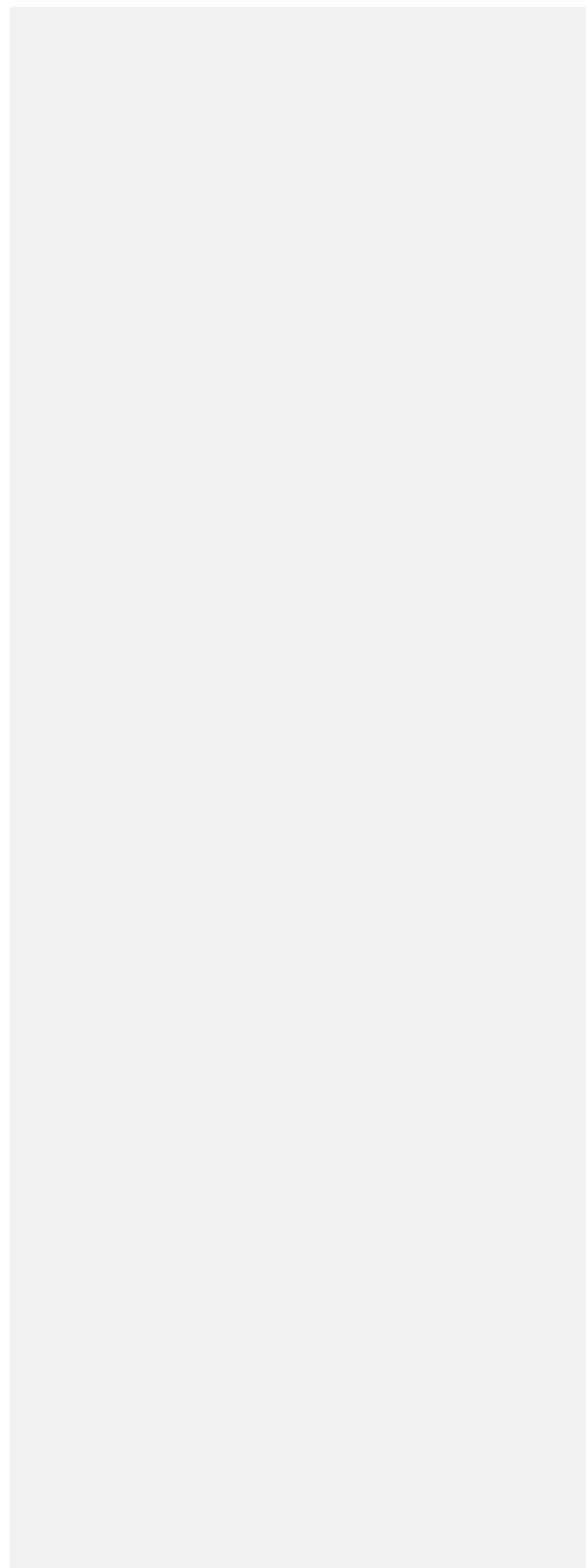
فهرست مطالب





بخش اول: شروع ماجراجویی!

در این بخش، مشعل را روشن می‌کنیم، با دنیای کد آشنا می‌شویم و یاد می‌گیریم چطور مثل یک ماجراجو، اولین معماها را حل کنیم.



فصل ۱: اولین قدم: آماده سازی محیط توسعه

مأموریت فصل: کارت معرفی دیجیتال برای ورود به انجمن ماجراجویان.

نقشه راه:

- چرا پایتون؟
- آماده کردن کوله پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین پایتون؛
- اولین طلسم جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه!
- پروژه: ساخت کارت معرفی دیجیتال مخصوص ماجراجویان با اسم خودت.

۱. چرا پایتون؟

صدها زبان برنامه‌نویسی وجود دارد، پس چرا همه دارند از پایتون حرف می‌زنند و چرا ما این ماجراجویی را با پایتون شروع می‌کنیم؟

۱-۱. یادگیری مثل چت کردن خیلی راحت است!

بزرگ‌ترین فرق پایتون با زبان‌هایی مثل C++ یا Java این است که قواعد نوشتاری^۱ خیلی ساده و روانی دارد. انگار داری به زبان انگلیسی دستور می‌دهی. به جای اینکه درگیر کلی علامت عجیب‌وغریب و قوانین سخت شوی، مستقیماً سراصل مطلب می‌روی و راه حل را پیاده می‌کنی.

۱-۲. با یک تیر، سه نشانه را هدف می‌گیری!

پایتون فقط برای شروع کردن عالی نیست؛ آن قدر قدرتمندست که بزرگ‌ترین شرکت‌های دنیا از آن برای ساختن آینده استفاده می‌کنند. با یادگیری پایتون، در واقع کلید ورود به سه تا از خفن‌ترین دنیاهای تکنولوژی را به دست می‌آوری:

هوش مصنوعی  تا حالا فکر کردی هوش مصنوعی‌هایی مثل ChatGPT چطور یاد می‌گیرند با تو حرف بزنند، یا ماشین‌های خودران چطور رانندگی یاد می‌گیرند؟ طلسم اصلی برای آموزش دادن به این هوش‌های مصنوعی، پایتون است. یعنی جادوگرهای دنیای تکنولوژی، با استفاده از پایتون به این ماشین‌ها یاد می‌دهند که چطور فکر کنند، الگوها را تشخیص دهند و تصمیم بگیرند. در واقع، پایتون زبان اصلی برای ساختن و تربیت کردن مغز متفکری‌ست که پشت این تکنولوژی‌های شگفت‌انگیز قرار دارد (البته زبان‌های دیگری هم هستند، ولی پایتون برای آموزش هوش مصنوعی، محبوب‌ترین است).

- **توسعه وب**^۳: آن بخش‌هایی که مغز متفکر سایت‌ها و اپلیکیشن‌های معروفی، مثل اینستاگرام، یوتیوب و کافه بازار هستند، با پایتون ساخته شده‌است (البته سایت‌های بزرگ از زبان‌های دیگر هم استفاده کردند و تنها پایتون نیست).

^۱ Syntax

^۲ AI

^۳ Web Development

- **علم داده^۴:** امروز، داده‌ها مثل گنج‌های پنهان‌اند. پایتون بهترین بیل و کلنگ برای کشف این گنج‌هاست. با آن می‌توانی حجم بالایی از اطلاعات را تحلیل کنی، الگوهای عجیب‌وغریب  ی و آینده را پیش‌بینی نمایی!

و البته خیلی کارهای جذاب دیگر هم می‌شود با پایتون انجام داد.

انتخاب پایتون مثل انتخاب یک چاقوی سوئیسی برای شروع ماجراجویی‌ست. تو با یک ابزار ساده، قدرتمند و همه‌کاره شروع می‌کنی که هم راه را برایت باز می‌کند و هم تو را به هر مقصدی که بخواهی می‌رساند.

پس کوله‌پشتی‌ت را آماده کن، چون قرارست با این زبان جادویی، برای تبدیل شدن به یک ماجراجوی واقعی در دنیای تکنولوژی، اولین قدم‌ها را برداری!

۲. آماده کردن کوله‌پشتی: نصب پایتون یا استفاده از آزمایشگاه آنلاین

هر ماجراجویی به ابزار نیاز دارد. ابزار اصلی ما، خود زبان پایتون است. برای استفاده از قدرت پایتون، دو راه اصلی پیش روی ماست: یکی اینکه کارگاه شخصی خودمان را بسازیم (نصب پایتون روی کامپیوتر یا لپ‌تاپ شخصیمان) و دومی اینکه از یک آزمایشگاه آنلاین جادویی و آماده استفاده کنیم (پایتونی که به‌صورت آنلاین ارائه می‌شود مناسب آنهایی‌ست که سیستم قوی ندارند و یا از گوشی و تبلت استفاده می‌کنند).

انتخاب با توست ماجراجو. تمام کدها برای دو مسیر در نظر گرفته شده، پس با خیال راحت هر کدام را که می‌خواهی انتخاب کن!

۲-۱. مسیر اول: ساخت کارگاه شخصی (نصب پایتون روی کامپیوتر و لپ‌تاپ)

این مسیر برای کسانی‌ست که دوست دارند همه‌چیز تحت کنترل خودشان باشد و ابزارهایشان را روی سیستم شخصی‌شان داشته باشند.

^۴ Data Science

○ قدم اول: نصب پایتون

۱. برای کاربران ویندوز

به وبسایت رسمی پایتون python.org برو. آنجا آخرین نسخه پایتون منتظر است. روی دکمه **Downloads** کلیک کرده و فایل نصبی^۵ برای ویندوز را دانلود کن.

مهم‌ترین نکته نصب: فایل نصبی را که اجرا کردی، به یک صفحه نصب می‌رسی. خیلی خیلی مهم است که در همان صفحه اول، تیک گزینه **Add Python to PATH** را بزنی. این کار مثل این است که به ویندوز بگویی: «هی، من یک ابزار جدید به اسم پایتون دارم، حواست بهش باشد» (اگه این کار را انجام ندهی، زمانی که کدهای پایتون را اجرا کنی، ارور^۶ پایتون را نمی‌شناسم می‌دهد. اگر یادت رفت این تیک را بزنی، دوباره مراحل را از اول برو و تیک را بزنی).

نصب نهایی: حالا روی **Install Now** کلیک کن و منتظر بمان تا نصب کامل شود. تمام! کارگاه شخصی تو آماده‌ست.

۲. برای کاربران مک

کاربران مک هم می‌توانند به سادگی از نصب‌کننده رسمی پایتون استفاده کنند.

سفر به منبع اصلی: به وبسایت رسمی پایتون python.org برو و وارد بخش **Downloads** شو. وبسایت به طور هوشمند سیستم عامل مک تو را تشخیص می‌دهد.

دریافت ابزار: روی دکمه دانلود آخرین نسخه برای macOS کلیک کن. یک فایل با پسوند *.pkg* یا *.dmg* دانلود می‌شود.

اجرای جادوی نصب: فایل نصب کننده را باز کن. این کار یک پنجره نصب استاندارد مک را باز می‌کند. مراحل بسیار ساده است؛ فقط کافی‌ست روی دکمه‌های **Continue**، **Agree** و در نهایت **Install** کلیک کنی. در این مرحله، مک از تو رمز عبور سیستم را می‌خواهد تا اجازه نصب را صادر کند.

پایان نصب: پس از چند لحظه، نصب کننده کارش را تمام می‌کند و پایتون به صورت کامل روی سیستم تو نصب می‌شود. حالا کارگاه شخصی تو روی مک آماده است!

^۵ Installer

^۶ Error

۳. برای کاربران لینوکس

لینوکس، سرزمین مورد علاقه برنامه‌نویس‌ها و پایتون معمولاً جزئی از خون این سیستم‌عامل است. اما باید مطمئن شویم که جدیدترین نسخه را داریم.

چک کردن نسخه: ترمینال را باز و این دستور را تایپ کن:

```
python3 --version
```

اگر نسخه‌ای که نشان می‌دهد از ۳.۶ بالاتر است، همه چیز آماده‌ست!

آپدیت کردن ابزار: اگر نسخه‌ات قدیمی‌ست یا اصلاً نصب نشده، با توجه به توزیع لینوکست (مثل اوبونتو، فدورا و...)، با یک دستور ساده آپدیتش کن. برای اکثر سیستم‌های مبتنی بر دبیان (مثل اوبونتو)، این دستور کافی‌ست:

```
sudo apt update  
sudo apt install python3
```

تمام! تو آماده‌ای.

○ قدم دوم: آماده کردن دفترچه جادو (محیط توسعه VS Code)

حالا که پایتون را نصب کردیم، به یک دفترچه جادویی برای نوشتن طلسم‌هایمان (کدها) نیاز داریم. بهترین انتخاب برای این کار، **Visual Studio Code** (یا به اختصار VS Code) است.

فکر کن VS Code یک دفترچه خفن است که کلمات جادویی را برایت رنگی می‌کند، اگر دستوری را اشتباه بنویسی زیرش خط می‌کشد و کمک می‌کند کدهایت را مثل یک ماجراجوی حرفه‌ای، تمیز و مرتب نگه داری.

دانلود دفترچه جادو: به وبسایت رسمی code.visualstudio.com برو. سایت به‌طور خودکار سیستم‌عامل تو (ویندوز، مک یا لینوکس) را تشخیص می‌دهد. روی دکمه بزرگ دانلود کلیک کن.

نصب ساده: فایل دانلودشده را اجرا و مراحل نصب را که بسیار ساده است، دنبال کن.

○ قدم سوم: افزونه شناسایی پایتون در دفترچه جادویی (VS Code)

برای اینکه دفترچه جادویی ما زبان پایتون را به بهترین شکل بفهمد، باید یک افزونه^۷ به آن اضافه کنیم؛ VS Code را باز کن. از نوار کناری سمت چپ، روی آیکونی که شبیه چهار مربع است کلیک کن (بخش Extensions). در کادر جست‌وجو، کلمه Python را تایپ کن.

اولین گزینه‌ای که توسط Microsoft ارائه شده را پیدا کن و روی دکمه آبی‌رنگ **Install** کلیک کن.

تبریک! حالا کارگاه شخصی تو با بهترین ابزارها آماده ماجراجویی است.

۲-۲. مسیر دوم: استفاده از آزمایشگاه جادویی آنلاین (Google Colab)

اگر حوصله نصب و درگیری با سیستم عامل را نداری، یا با گوشی و تبلت این ماجراجویی را دنبال می‌کنی، یا سیستم‌ات خیلی قوی نیست، این مسیر برای تو ساخته شده است!

گوگل کولب^۸ یک دفترچه یادداشت آنلاین و هوشمندست که پایتون از قبل روی آن نصب شده و فقط منتظر دستورات جادویی توست.

○ چرا خفن است؟

- بدون نیاز به نصب: فقط با یک مرورگر و اکانت گوگل (رایگان) کارت راه می‌افتد.
- همیشه در دسترس: از هرجایی (کافه، کتابخانه، اتوبوس) و با هر وسیله‌ای (لپ‌تاپ، تبلت، گوشی) به کدهایت دسترسی داری.
- قدرت ابری: انگار داری با کامپیوترهای قدرتمند گوگل کار می‌کنی، پس نگران ضعیف بودن سیستم نباش (در عمل کدهایت روی سرورهای گوگل اجرا می‌شوند).

○ چطور شروع کنیم؟

فقط کافی‌ست به وب‌سایت colab.research.google.com بروی، با اکانت گوگل وارد شوی و روی **New notebook** کلیک کنی. یک صفحه جدید باز می‌شود که می‌توانی اولین کدهای پایتونت را همان‌جا بنویسی و اجرا کنی. به همین سادگی و کاملاً رایگان! (البته محدودیت‌هایی دارد، مثل تعداد ساعت و سخت‌افزار که افزایش آنها هزینه دارد، ولی در این کتاب اصلاً نیازی به اکانت پרוی گوگل کولب نداریم و با اکانت رایگان می‌توانی تا انتهای کتاب را بیایی).

^۷ Extension

^۸ Google Colab

خب ماجراجو، کوله‌پشتیت را انتخاب کردی؟ ابزارت را آماده کن که می‌خواهیم اولین طلسم جادویی را اجرا کنیم!

۳. اولین طلسم جادویی: اجرای `print("Hello, World!")` و دیدن نتیجه!

تبریک! کوله‌پشتیت آماده است و ابزارهایت را انتخاب کرده‌ای. اکنون زمان این است که اولین طلسم جادویی خود را یاد بگیری و آن را اجرا کنی. این لحظه، لحظه‌ای فراموش‌نشدنی و خاص است؛ لحظه‌ای که برای اولین بار به ماشین دستور می‌دهی و او به تو پاسخ می‌دهد.

این طلسم جادویی، `print` نام دارد. کارش خیلی ساده است: هر چیزی که داخل پرانتز و بین دو علامت کوتیشن^۹ (نقل قول) " " به آن بدهی، روی صفحه نمایش نشان می‌دهد. مثل: `print("Hi")` بیایید با هم این طلسم را اجرا کنیم!

۳-۱. اگر از مسیر اول (کارگاه شخصی و VS Code) می‌آیی:

برای اینکه ماجراجویی‌هایمان مرتب باشد، اول یک پوشه^{۱۰} برای پروژه‌مان می‌سازیم.

ساخت پوشه پروژه: روی دسکتاپ یا هر جای دیگری که دوست داری، یک پوشه جدید بساز و اسم آن را `my_project` بگذار. تمام طلسم‌ها، گنج‌ها و نقشه‌های ما در این پوشه نگهداری خواهند شد.

باز کردن دفترچه جادو (VS Code): برنامه VS Code را که قبلاً نصب کردی، باز کن.

باز کردن پوشه پروژه در VS Code: از منوی بالا، روی `File` کلیک کرده و گزینه `Open Folder` را انتخاب کن. حالا به همان پوشه پروژه که ساختی برو و آن را انتخاب کن. با این کار، تمام تمرکزش را روی این پوشه و پروژه تو می‌گذارد.

ساخت اولین فایل پایتون: در سمت چپ VS Code، جایی که اسم پوشه پروژه را می‌بینی، یک آیکون کوچک شبیه به یک برگه با علامت + وجود دارد. روی آن کلیک کن و اسم فایل جدید را `project.py` بگذار. پسوند `.py` به VS code می‌گوید که این یک فایل پایتون است! (حتماً پسوند فایل را `.py` بگذار. این خیلی مهم است! وگرنه دیگر VS code فایلت را درست نمی‌شناسد. این قاعده به‌طور کلی وجود دارد که پسوند فایل، مثل `.py` یا `.txt` یا `.png`، به ما می‌گویند که محتوای این فایل قرارست چه باشد).

نوشتن طلسم جادویی: حالا در صفحه سفیدی که باز شده، اولین طلسم جادویی خود را با دقت تایپ کن (خیلی به علامت‌های " و فاصله‌ها و... دقت کن، چون در برنامه‌نویسی اگر حتی یک نقطه را

^۹ Quotation

^{۱۰} Folder

اشتباه بگذاری، کدت کار نمی‌کند (اعدادی که در کنار کدها توی این کتاب می‌بینی، صرفاً نشان‌دهنده شماره خط است و جزوی از کد نیستند و نباید آنها را بنویسی):

```
1. print("Hello, World!")
```

اجرای طلسم: بخش هیجان‌انگیز ماجرا فرا رسیده است. باید ترمینال یا همان اتاق فرمان را باز کنیم تا طلسم خود را در آن بخوانیم.

از منوی بالای VS Code، روی Terminal کلیک کن و New Terminal را انتخاب کن.

راه میانبر و سریع‌تر: می‌توانی دکمه‌های ترکیبی ~ + Ctrl (کلید ~ معمولاً کنار عدد ۱ روی کیبورد است) را فشار دهی تا ترمینال فوراً باز شود. در مک، این میانبر ~ + Cmd است.

خواندن دستور نهایی: در ترمینالی که پایین صفحه باز شده، این دستور را تایپ کن و کلید Enter را بزن (به فاصله و املاي کلمات دقت کن و همچنین اگر اسم فایل پایتون را چیز دیگری گذاشتی، آن را اینجا و به صورت کامل با پسوند *.py* بنویس):

```
python project.py
```

نکته برای ماجراجویان مک و لینوکس: به جای دستور بالا، python3 project.py را امتحان کنید.

و ناگهان... جادو اتفاق می‌افتد!

درست در خط بعدی ترمینال، این پیام ظاهر می‌شود:

```
Hello, World!
```

تبریک! تو همین الان اولین کد خودت را اجرا کردی. تو به کامپیوتر دستور دادی و او از تو پیروی کرد. این اولین قدم تو برای تبدیل شدن به یک جادوگر قدرتمند در دنیای کدنویسی است. این لحظه را جشن بگیر!

۳-۲. اگر از مسیر دوم (آزمایشگاه جادویی Google Colab) می‌آیی:

کار تو بسیار ساده‌تر است! آزمایشگاه آنلاین همه‌چیز را برایت آماده کرده است.

ورود به آزمایشگاه: در همان نوت‌بوک جدیدی که در Colab باز کردی، یک سلول کد خاکستری‌رنگ با

یک دکمه اجرا (▶) در کنارش می‌بینی (به اینها سلول کد می‌گویند و می‌توانی در آنها کد خودت را

بنویسی و با دکمه پلی "کنارش کدت اجرا می‌شود و اگر نبود، می‌توانی از دکمه بالا، گزینه Code را بزنی تا یک سلول کد جدید ایجاد شود). یادت باشد هر کدی که در سلولی وارد کردی، باید دکمه اجرا را بزنی. نوشتن **طلسم جادویی**: داخل این سلول، طلسم جادویی را تایپ کن (اعدادی که در کنار کدها توی این کتاب می‌بینی، صرفاً نشان‌دهنده شماره خط است و جزوی از کد نیستند و نباید آنها را بنویسی):

```
1. print("Hello, World!")
```

اجرای طلسم: حالا کافی‌ست روی همان دکمه اجرا (▶) کلیک کنی یا کلیدهای Shift + Enter را با هم فشار دهی.

بلافاصله زیر سلول کد، نتیجه جادوی خود را خواهی دید:

```
Hello, World!
```

فوق‌العاده بود، نه؟ تو اولین کد خودت را روی سرور گوگل اجرا کردی! ماجراجویی رسماً آغاز شد!

۴. پروژه: ساخت کارت ویزیت دیجیتال^{۱۲} - 100 xp

ماجراجوی عزیز، به انجمن خوش آمدی! تو اولین طلسم جادویی خودت را یاد گرفتی. حالا وقت آن است که با استفاده از همین قدرت، هویت خودت را رسماً به انجمن ماجراجویان پایتون اعلام کنی. ما در این پروژه یک کارت ویزیت دیجیتال خواهیم ساخت تا همه بدانند با چه ماجراجوی جدیدی آشنا شده‌اند.

۴-۱. تصویر نهایی: این کارت چه شکلی است؟

قبل از شروع، بیا نتیجه را ببینیم. وقتی برنامه را اجرا کنی، قرار نیست یک کاغذ از پرینتر بیرون بیاید! بلکه در صفحه سیاه ترمینال (یا خروجی Colab)، مشخصات تو، خطبه خط و خیلی مرتب ظاهر می‌شوند. چیزی شبیه به این:

```
Name: Mahdi Ghaemi
Title: Python World Explorer
Contact: adventurer@example.com
Motto: Ready to solve the first puzzle!
=====
```

¹¹ Play

¹² The Digital ID Card

۴-۲. نقشه و طرح اولیه

در اولین پروژه، ما باید ابزارهایمان را بشناسیم. اینها آچار و پیچ‌گوشتی‌های تو برای این مأموریت‌اند:

۱. **طلسم چاپگر print() و راز پرانتزها:** فکر کن **print** یک فرمانده است که به کامپیوتر

فرمان می‌دهد: **نمایش بده!** اما کامپیوتر می‌پرسد: چه چیزی را؟ اینجا است که پرانتزها () وارد می‌شوند. پرانتزها مثل یک ظرف یا سینی‌اند. هر چیزی که داخل این ظرف بگذاری، تحویل پرنتر داده می‌شود.

• **print** دستور (انجام بده):

• () ظرف ورودی (روی چه چیزی انجام بده؟).

۲. **مرزهای جادویی کوتیشن‌ها (" "):** چرا متنی که می‌نویسیم را داخل " " می‌گذاریم؟ پایتون باید بفهمد کجا دستور تمام شده و کجا پیام متنی شروع شده است. به مثال زیر دقت کن:

```
1. print("print print")
```

• **print** اول: دستور پایتون (اجرا می‌شود).

• **"print print"** دوم: فقط یک متن ساده است (چاپ می‌شود). اگر " نباشد، پایتون گیج می‌شود و فکر می‌کند کلمه دوم هم یک دستور است!

۳. **قانون آبشار (جریان اجرا^{۱۳}):** پایتون کدها را مثل یک آبشار می‌خواند؛ از بالا به پایین، خط به خط. یعنی اول خط ۱ را کامل اجرا می‌کند، بعد می‌رود سراغ خط ۲. پایتون هرگز نمی‌پرد و هرگز دو خط را هم‌زمان اجرا نمی‌کند. این ترتیب به ما اجازه می‌دهد داستانمان را مرحله به مرحله تعریف کنیم.

۴. **یادداشت‌های نامرئی # کامنت‌ها:** ماجراجویان باهوش همیشه یادداشت برمی‌دارند. در پایتون، اگر اول خط، علامت # بگذاری، آن خط نامرئی می‌شود! یعنی پایتون آن خط را نمی‌بیند و اجرا نمی‌کند. پس فایده‌اش چیست؟ این یادداشت‌ها برای تو و بقیه انسان‌هاست تا یادتان نرود کد چه کاری انجام می‌دهد.

۴-۳. چالش‌های فکری

قبل از نوشتن کد اصلی، به این سؤال‌ها فکر کن:

[GG1] Commented: توی این پاراگراف `print` هم طلسم گفته شده، هم فرمانده و هم دستور. به کم گیج کننده ست. به جای این مثال، پیشنهاد میشه: «فکر کن `print` یک ورد جادویی ست که به کامپیوتر می‌گوید: نمایش بده»
در جمله بعدی هم به جای ظرف میشه از «جام» استفاده کرد که به کم به فضای جادو نزدیک‌تره.

[GG2] Commented: اینجا میشه بعد از `PRINT` یا پرانتزها، یک : هم گذاشت.

¹³ Control Flow

- چالش ۱. ظرف خالی: اگر بنویسیم `print()` و داخل پرانتز چیزی نگذاریم، چه اتفاقی می‌افتد؟
(پاسخ: یک خط خالی چاپ می‌شود- مثل این است که یک کاغذ سفید به پرینتر بدهی).
- چالش ۲. شکستن مرز: اگر بنویسیم `print("Hello")` (یعنی کوتیشن دوم را فراموش کنیم)، پایتون چه واکنشی نشان می‌دهد؟ (پاسخ: خطای `Syntax Error` می‌دهد، چون منتظر است ببیند متن کجا تمام می‌شود).

۴-۴. کد نهایی پروژه (The Spell)

حالا وقت خلق کردن است. کد زیر را در فایل `project.py` بنویس.

مهم: اطلاعات داخل کد (مثل اسم من) را پاک و مشخصات واقعی خودت را جایگزین کن و به کامنت‌های انگلیسی دقت کن تا ببینی چطور از `#` استفاده کرده‌ایم.

```
1. # --- Adventurer's ID Card ---
2.
3. # Line 1: Displays the adventurer's name
4. # (Replace [Your Name] with your real name)
5. print("Name: Mahdi Ghaemi")
6.
7. # Line 2: Your chosen title in this journey
8. print("Title: Python World Explorer")
9.
10. # Line 3: A way for others to contact you
11. print("Contact: mahdighaemi123@gmail.com")
12.
13. # Line 4: Your personal motto (Slogan)
14. print("Motto: Don't touch the working code!")
15.
16. # Line 5: A border to make it look cool
17. print("=====")
```

Commented [GG3]: این عبارت، معادل انگلیسی «کد نهایی» نیست، پس در فوت نوت نمیره. اما در جهت اهداف جادویی لازمه. موافقید که حفظ بشه در همین جا توی پرانتز؟

Commented [GG4]: دقیقاً با پیش نمایش کد در چند صفحه قبل مطابقت ندارد. لطفاً بررسی شود

Commented [GG5]: با خروجی از پیش نمایش داده شده مطابقت ندارد

اجرا کن و لذت ببر! دستور اجرا را بزن. دکمه ▶ یا `python project.py` در ترمینال. پایتون دقیقاً به همان ترتیبی که گفتی، کارت تو را ساخت.

۴-۵. جشن پیروزی و تحلیل

تبریک می‌گویم! تو نه تنها اولین کد خودت را اجرا کردی، بلکه اولین محصول نرم‌افزاریت را هم ساختی. این کارت معرفی، مدرک ورود تو به دنیای شگفت‌انگیز برنامه‌نویسی است.

مأموریت جانبی: از خروجی کارت عکس بگیر و در شبکه‌های اجتماعی با هشتگ `#codewithmahdi` و `#pythonadventure` به اشتراک بگذار. با این کار، ما می‌فهمیم که یک ماجراجوی جدید و با استعداد به جمع ما اضافه شده‌است!

Commented [GG6]: اشتراک گذاری نتیجه. اگر لازم است فکری برای نحوه اشتراک گذاری شود

در فصل‌های بعد باهم یاد می‌گیریم کارهای باحال‌تری انجام دهیم و چیزهای بیشتری بسازیم.

| 🌸 ارتقاء لول: 🌸 |

۱ → ۲

مهارت‌های جدید:

◆ دستور Print

له نام باستانی: آوای آفرینش.

توضیح: سکوت جهان دیجیتال شکسته شد. اکنون کلمات تو بر صفحه سیاه هستی نقش می‌بندند.

◆ سطح جدید: H

توضیح: تبریک! به سطح جدیدی رسیدی.

فصل ۲: تبدیل شدن به کارآگاه کد

مأموریت فصل: شکار اولین باگ.

نقشه راه:

- ذهنیت یک ماجراجو: برنامه نویسی یعنی حل مسئله، نه حفظ کردن کد!
- جعبه ابزار کارآگاه:
- Stack Overflow: کتابخانه بزرگ برای تمام جادوگران دنیا.
-  دستیارهای هوشمند (AI): چطور از غول های چراغ جادو کمک بگیریم؟
- هنر پرسیدن سؤال خوب.
- پروژه: یک کد نفرین شده به شما داده می شود. با ابزارهای جدیدتان، خطای آن را پیدا نموده و درستش کنید.

۱. ذهنیت یک ماجراجو: برنامه‌نویسی یعنی حل مسئله، نه حفظ کردن کد!

اولین و مهم‌ترین قانونی که هر ماجراجوی کهنه‌کاری می‌داند این است: برنامه‌نویسی یعنی حل مسئله، نه حفظ کردن کد!

بسیاری از کسانی که تازه شروع کرده‌اند، فکر می‌کنند که باید صدها دستور و طلسم جادویی مانند print را حفظ کنند. اما اصل ماجرا این نیست. اصل ماجرا در تفکر منطقی توست.

وقتی کدت کار نمی‌کند یا با یک پیام خطای ترسناک قرمزرنگ (که به زودی با آنها آشنا می‌شویم) برخورد می‌کنی، نترس! این یک معماست. کامپیوتر، برخلاف انسان‌ها، موجودی کاملاً منطقی است. او دقیقاً کاری را می‌کند که به او می‌گویی. اگر نتیجه اشتباه است، یعنی ما دستور را به شکلی نامفهوم یا اشتباه به او داده‌ایم.

به این دستورات اشتباه، باگ^۱ یا طلسم معیوب می‌گوییم و مأموریت ما در این فصل، رفع نفرین (دیباگینگ^۲) از آنهاست.

پس ذهنیت خود را آماده کن: تو یک کارآگاه هستی. هر خطا^۳ یک سرخ است. هر باگ، یک چالش جذاب است که منتظر توست تا آن را حل کنی.

۲. جعبه ابزار کارآگاه

هر کارآگاهی به ابزار نیاز دارد. برای شکار باگ‌ها، لازم نیست همه چیز را از صفر اختراع کنیم. ما دو ابزار فوق‌العاده قدرتمند در کوله‌پشتی خود داریم که باید طرز استفاده از آنها را یاد بگیریم.

۲-۱. Stack Overflow: کتابخانه بزرگ برای تمام جادوگران دنیا

تصور کن یک کتابخانه جادویی بی‌انتهای وجود دارد که در آن، هر جادوگر و ماجراجویی که تا به حال با یک کد نفرین شده روبه‌رو شده، مشکل و راه‌حلش را در آن ثبت کرده‌است.

آن کتابخانه، Stack Overflow نام دارد.

این یک وب‌سایت پرسش و پاسخ برای برنامه‌نویسان است. به احتمال ۹۹ درصد، هر مشکلی که تو در این ماجراجویی با آن روبه‌رو می‌شوی، قبلاً صدها نفر دیگر هم با آن مواجه شده‌اند و بهترین جادوگران

^۱ Bug

^۲ Debugging

^۳ Error

دنیا به آن پاسخ داده‌اند. یادگیری جست‌وجو کردن در این کتابخانه، اغلب مهم‌تر از حفظ کردن هر طلسمی است (برای استفاده، کافی‌ست پیغام خطایی که با آن مواجه شدی را در گوگل سرچ کنی و حتماً در میان نتایج جست‌وجو، Stack Overflow را می‌بینی).

۲-۲. دستیارهای هوشمند (AI): چطور از غول‌های چراغ جادو کمک بگیریم؟

حالا تصور کن غول‌هایی وجود دارند که می‌توانند به ما در نوشتن طلسم‌های بهتر کمک نمایند، کدهای ما را توضیح دهند و حتی باگ‌های آن را پیدا کنند!

ابزارهایی مثل Gemini و ChatGPT دقیقاً همین کار را می‌کنند. آنها دستیارهای هوشمند تو هستند. می‌توانی کد باگ‌دار خود را بهشان نشان بدهی و بپرسی: «کجای این کد اشتباه‌ست؟». آنها هم با صبر و حوصله به تو کمک می‌کنند که مشکل را پیدا کنی.

هنر پرسیدن سؤال خوب

داشتن جعبه‌ابزار کافی نیست؛ باید بلد باشی چطور از آن استفاده کنی.

چه در Stack Overflow و چه از دستیار هوشمند، خوب پرسیدن یک هنر است. یک ماجراجوی تازه‌کار می‌گوید: «کمک! کد من کار نمی‌کند!» و هیچ‌کس نمی‌تواند به آن کمک کند.

اما یک کارآگاه حرفه‌ای می‌گوید: «من در حال اجرای یک طلسم print در فایل *project.py* هستم. انتظار داشتم کارت ویزیت را چاپ کند، اما در عوض با این پیام خطای قرمز مواجه شدم: `SyntaxError: EOL while scanning string literal`. این هم کدی‌ست که نوشته‌ام. به نظرتان مشکل از کجاست؟».

همیشه سه چیز را در سؤال بیاور:

۱. چه کار می‌خواستی بکنی؟ (نتیجه مورد انتظار؟)
 ۲. چه مشکلی پیش آمد؟ (پیام خطا دقیقاً چه بود؟ آن را کپی کن)؛
 ۳. کدی که نوشتی چه بود؟ (کد خودت را نشان بده)؛
- با این سه مورد، هرکسی می‌تواند به تو در شکار باگ کمک کند.

۳. پروژه: شکار باگ^۴ در کد نفرین شده - 100 xp

آماده‌ای اولین شکار را انجام دهی کارآگاه؟ در این مأموریت، ما قرار نیست چیزی را از صفر بسازیم. در عوض، یک تکه کد پایتون داریم که قرار بود کارت ویزیت یک ماجراجو را چاپ کند، اما توسط یک جادوگر خبیث نفرین شده و کار نمی‌کند. پایتون با کسی شوخی ندارد؛ اگر قوانین نگارش^۵ را رعایت نکنی، خطای قرمز نشانت می‌دهد. مأموریت تو این است که کد را دیباگ کنی (منظور از باگ، مشکل و خطاست؛ پس به رفع خطا و مشکل، دیباگ می‌گوییم).

۳-۱. تصویر نهایی: کارت ویزیت سالم

قبل از اینکه وارد صحنه جرم شویم، باید بدانیم نتیجه سالم چه شکلی است. اگر کد را درست بنویسی، خروجی باید دقیقاً این شکلی، تمیز و بدون هیچ نوشته قرمزی در ترمینال چاپ شود:

```
Name: Mahdi ghaemi
Title: World Ender
Motto: Brave and Bold!
=====
```

۳-۲. نقشه کارآگاه (چگونه باگ شکار کنیم؟)

یک برنامه‌نویس حرفه‌ای وقتی ارور می‌بیند، نمی‌ترسد؛ بلکه کلاه کارآگاهیش را سرش می‌گذارد. برای رفع طلسم، باید این ۳ مرحله را طی کنی:

۱. **صحنه جرم را بررسی کن (اجرا):** کد خراب را کپی و اجرا کن. پیام خطای قرمزرنگی که پایتون در پایین صفحه (ترمینال) به تو نشان می‌دهد، دشمن تو نیست؛ سرخ توست!

۲. **آدرس دقیق را پیدا کن:** پایتون معمولاً در پیام خطا به تو می‌گوید که در کدام خط (Line x) گیج شده‌است. مستقیماً به همان خط برو.

۳. **بررسی مزنون‌ها (قوانین نگارش):** حالا باید مثل یک معلم سخت‌گیر، گرامر کد را چک کنی:

- **مرزهای جادویی:** آیا هر متنی که شروع شده (")، پایان هم دارد (")?
- **ظروف جادویی:** آیا هر پرانتزی که باز (شده، بسته (هم شده‌است ؟

^۴ Bug Hunting

^۵ Syntax

- **رنگ کد:** در محیط‌های کدنویسی (IDE)، رنگ متن‌ها باید متفاوت از کدها باشد. اگر رنگ‌ها قاطی شده‌اند، یعنی یک جای کار می‌لنگد.

۳-۳. چالش‌های فکری

کد طلسم‌شده زیر را در فایل *project.py* کپی کن (بچه‌های Colab: در یک سلول کپی کنید). این کد پر از اشکال است! قبل از دیدن جواب، سعی کن با ذره‌بین خودت پیدایشان کنی:

```
1. # --- Cursed Code (DO NOT RUN... yet!) ---
2. print("Name: Mahdi ghaemi)
3. print("Title: World Ender"
4. print("Motto: "Brave and Bold!")
5. print(=====)
```

- **چالش ۱. خط دوم (فرار متن):** در خط ۲، متن `Name: ...` شروع شده، اما مرز پایانی ندارد. پایتون از کجا بفهمد متن تمام شده است؟ (راهنمایی: کوتیشن " گم شده است).
- **چالش ۲. خط سوم (ظرف باز):** در خط ۳، ما دستور `print` را باز کردیم، اما درش را نبستیم! (راهنمایی: پرانتز گم شده است).
- **چالش ۳. خط پنجم (اشتباه هویت):** در خط آخر، ما می‌خواستیم یک خط جداکننده `=====` چاپ کنیم. اما چون دورش " نگذاشتیم، پایتون فکر می‌کند اینها دستورات ریاضی‌اند و گیج می‌شود!

جعبه ابزار کمکی: اگر روزی با خطایی روبه‌رو شدی که آن را نفهمیدی (مثلاً `SyntaxError`)، متن خطا را در گوگل کپی / پیست کن و کلمه `Stack Overflow` را ته آن بنویس. آنجا کتابخانه بزرگ جادوگران است که قبلاً هزاران نفر مشکل تو را داشته و راه حلش را پیدا کرده‌اند.

۳-۴. کد نهایی و درمان شده (The Cure)

حالا وقت باطل کردن طلسم است. تغییرات را در کد اعمال کن تا به شکل زیر دربیاید:

```
1. # --- My ID Card (Cure applied) ---
2.
3. # 1. Fixed the missing quote at the end
4. print("Name: Mahdi ghaemi")
5.
6. # 2. Fixed the missing parenthesis
7. print("Title: World Ender")
8.
9. # 3. Fixed the messy quotes inside the string
10. print("Motto: Brave and Bold!")
11.
12. # 4. Added quotes to make it a string, not code
```

```
13. print("=====")
```

اجرا کن و نفس راحت بکش! الان باید کارت ویزیت به‌زیبایی چاپ شود. به این حس خوبی که الان داری، لذت دیباگ کردن می‌گویند (برای من لذتش شبیه به درآوردن سوزنی است که توی پایم رفته بود).

۳-۵. جشن پیروزی و تحلیل

تبریک می‌گویم کارآگاه! تو الان رسماً اولین باگ خودت را شکار کردی. یادت باشد، حتی بهترین برنامه‌نویسان دنیا هم هر روز با ارورهای قرمز مواجه می‌شوند. تفاوت آنها با بقیه این است که ارور را شکست نمی‌دانند، بلکه آن را یک معما می‌بینند که باید حل شود.

کوله‌پشتی تو برای ماجراجویی بعدی سنگین‌تر و ارزشمندتر شد. در فصل بعد، ابزارهای جادویی جدیدی برای کار با اعداد خواهیم ساخت!

🌟 | ارتقاء لول: 🌟 |

۲ → ۳



مهارت‌های جدید:

◆ مهارت Debugging

له نام باستانی: شکار سایه‌ها.

توضیح: هیچ طلسمی بی‌نقص نیست. هنر تو یافتن شکاف‌ها و ترمیم آنهاست.

◆ جست‌وجو در StackOverflow

له نام باستانی: احضار خرد جمعی.

توضیح: اتصال به ذهن هزاران جادوگر دیگر برای یافتن پاسخ.

◆ سطح جدید: G

توضیح: تبریک! به سطح جدیدی رسیدی.

فصل ۳: جعبه‌های جادویی: ماشین حساب جادویی

مأموریت فصل: ساخت یک ماشین حساب جادویی برای محاسبه و تقسیم غنائم بین اعضای گروه.

نقشه راه:

- جعبه‌های جادویی (متغیرها): چطور غنائم و اطلاعات را در حافظه نگه داریم؟
- آشنایی با غنائم: انواع داده‌های پایه (Booleans و Strings, Floats, Integers).
- جادوی ریاضی: کار با عملگرهای +، -، * و / روی اعداد.
- جادوی متن: چطور طلسم + برای متن‌ها (Strings) متفاوت عمل می‌نماید؟
- پروژه: ساخت یک ماشین حساب که غنائم (اعداد ثابت) را محاسبه و نتایج را چاپ می‌کند.

۱. جعبه‌های جادویی (متغیرها)

وقتی در ماجراجویی خود یک سکه طلا یا یک معجون سلامتی پیدا می‌کنی، آن را روی زمین رها نمی‌کنی تا گم شود! بلکه در یک جعبه امن می‌گذاری تا بعداً از آن استفاده کنی.

در برنامه‌نویسی، به این جعبه‌های جادویی، متغیر^۱ می‌گوییم.

متغیرها، مکان‌هایی در حافظه رم^۲ کامپیوترند که ما داده‌هایمان را در آنها ذخیره می‌کنیم. این کار باعث می‌شود بتوانیم بعداً به آن داده‌ها دسترسی داشته باشیم. این داده‌ها موقتی‌اند و وقتی برنامه تمام شود (یا کامپیوتر خاموش شود)، از بین می‌روند (در فصل‌های بعد با اینکه چه‌طور به‌صورت دائمی اطلاعات را ذخیره کنیم بیشتر آشنا می‌شویم).

ساختن یک متغیر، مثل برجسب زدن به یک جعبه است. ما از علامت = (که به آن عملگر تخصیص می‌گوییم) برای ریختن چیزی در جعبه استفاده می‌کنیم (یادت نرود وقتی # را ابتدای یک خط می‌گذاریم، یعنی آن خط توضیح است و پایتون آن را اجرا نمی‌کند):

```
1. # "Mahdi Ghaemi" is the content inside the box
2. # adventurer_name is a label
3. adventurer_name = "Mahdi Ghaemi"
4.
5. # gold_coins is a label
6. # 50 is the content inside the box
7. gold_coins = 50
```

حالا، هر وقت که بخواهیم، می‌توانیم محتوای داخل جعبه را با استفاده از print و نام آن جعبه ببینیم (دقت کن که نام متغیر نباید داخل " باشد، وگرنه با متن اشتباه گرفته می‌شود):

```
1. # Let's make some magic boxes
2. adventurer_name = " Mahdi Ghaemi"
3. gold_coins = 50
4. health_points = 100
5.
6. # Now let's print the content of the boxes
7. print("Adventurer's Name:")
8. print(adventurer_name) # Python prints the box content, not its name
9.
10. print("Gold Coins:")
11. print(gold_coins)
12.
13. print("Health Points:")
14. print(health_points)
```

^۱Variable

^۲Random-access Memory (RAM)

اجرا کن و نتیجه را ببین! وقتی این کد را اجرا کنی، کامپیوتر به جای چاپ کلمه "gold_coins"، محتوای داخل آن جعبه (یعنی ۵۰) را چاپ می‌کند.

۱-۱. قدرت متغیرها: تغییرپذیری

جادوی واقعی متغیرها این است که محتوای آنها می‌تواند تغییر کند.

```
1. # At the start of the adventure, we have 10 coins
2. gold_coins = 10
3. print("Current gold coins:")
4. print(gold_coins)
5.
6. # We defeat a giant and get 20 new coins
7. # We replace the old content of the box with the new value
8. gold_coins = 20
9. print("Gold coins after defeating the giant:")
10. print(gold_coins)
11.
12. # We can even do calculations on the box itself
13. # We spend 5 coins
14. gold_coins = gold_coins - 5
15. print("Remaining gold coins:")
16. print(gold_coins) # Output will be 15
```

این قابلیت برای ماجراجویی ما خیلی مهم است!

۲. آشنایی با انواع داده‌های پایه^۳

متغیرها مثل جعبه‌های جادویی‌اند که می‌توانند چیزهای مختلفی را داخل خودشان نگه دارند. اما توی این ماجراجویی یک قانون خیلی مهم وجود دارد:

نوع چیزی که داخل جعبه می‌گذاری، مشخص می‌کند چه کارهایی می‌توانی با آن انجام بدهی. مثلاً توی دنیای واقعی نمی‌توانی یک شمشیر را با سکه طلا جمع بزنی! شمشیر شبیه متن است و سکه طلا شبیه عدد. هرکدام جادوی مخصوص خودشان را دارند. به این نوع داده‌ها، داده‌های پایه می‌گوییم.

اینها ساده‌ترین و اصلی‌ترین نوع داده‌ها هستند و از همان اول در خود پایتون ساخته شده‌اند. پایتون خودش می‌فهمد که:

- چه چیزی عدد است؛
- چه چیزی متن است.

Commented [GG8]: به نظرم بهتره که اینجا از کوتیشن استفاده نشه که با رشته نمایش داده شده خط ۱۰ اشتباه گرفته نشه. درسته؟

^۳ Primitive Types

و لازم نیست تو این چیزها را به او یاد بدهی.

در ماجراجویی پایتون، ما با چهار نوع داده اصلی سروکار داریم:

۱. اعداد صحیح^۴

۲. اعداد اعشاری^۵

۳. رشته^۶

۴. بولین^۷

۲-۱. اعداد صحیح یا Integer

این تئجرها در واقع اعداد کامل و صحیح‌اند. می‌توانند مثبت، منفی یا صفر باشند. هر وقت چیزی را می‌شماری (تعداد تیر، تعداد دشمن یا تعداد قدم) از این نوع استفاده می‌کنی.

```
1. age = 21
2. score = -20
3. count = 0
```

۲-۲. اعداد اعشاری یا Float

فلوت‌ها در واقع اعداد اعشاری‌اند؛ هر وقت به دقت نیاز داری (مثل محاسبه میزان دما یا درصد سلامتی) از این نوع استفاده می‌کنی.

```
1. pi = 3.14
2. temp = -273.0
3. percent = 99.0
```

نکته: عدد 22 عدد صحیح و 22.0 عدد اعشاری محسوب می‌شود. پایتون اعدادی که اعشار 0 دارند را به صورت خودکار به صحیح تبدیل نمی‌کند و به همان شکلی که هستند نگه می‌دارد.

^۴ Integer

^۵ Float

^۶ String

^۷ Boolean

۳-۲. رشته یا String

رشته‌ها در واقع متن هستند. هر چیزی که داخل علامت کوتیشن (" یا ') قرار بگیرد، یک متن یا String است. نام‌ها، پیام‌ها و... همه از این نوع‌اند.

```
1. name = "mahdi"
2. message = "Oops!"
```

۴-۲. بولین یا Boolean

بولین‌ها فقط دو حالت دارند: True (درست) یا False (نادرست). بولین‌ها برای تصمیم‌گیری استفاده می‌شوند. (آیا در باز است؟ True. آیا من زنده‌ام؟ False). ما در فصل بعد خیلی بیشتر از آنها استفاده می‌کنیم.

```
1. is_up = True
2. is_down = False
```

۳. جادوی ریاضی: کار با عملگرها

حالا که عدد صحیح و عدد اعشاری را می‌شناسیم، بیا با آنها محاسبات انجام دهیم. پایتون چهار عملگر اصلی ریاضی را بلد است:

- + (جمع)
- - (تفریق)
- * (ضرب)
- / (تقسیم)

بیا اینها را با متغیرها امتحان کنیم. ما ۲ صندوقچه با ۵۰ سکه در یکی و ۷۵ سکه در دیگری پیدا کردیم. می‌خواهیم این سکه‌ها را بین ۴ نفرمان تقسیم کنیم. به هر نفر چقدر می‌رسد؟

```
1. # We found the loot
2. gold_coins_chest1 = 50
3. gold_coins_chest2 = 75
4. adventurers_count = 4
5.
6. # --- Performing the magic of math ---
7.
8. # 1. Add the loot from both chests
9. total_gold = gold_coins_chest1 + gold_coins_chest2
10.
11. # 2. Divide the loot among the party members
12. gold_per_adventurer = total_gold / adventurers_count
13.
```

```

14. # --- Displaying the results ---
15. print("Total gold found:")
16. print(total_gold) # Output: 125
17.
18. print("Share per adventurer:")
19. print(gold_per_adventurer) # Output: 31.25

```

نکته مهم: دیدی که حاصل تقسیم ۱۲۵ بر ۴ شد ۳۱٫۲۵؟ در پایتون، عملگر / همیشه نتیجه را به صورت عدد اعشاری برمی‌گرداند تا هیچ بخشی از غنائم در تقسیم از بین نرود!

۴. جادوی متن: چسباندن متن‌ها به هم

یک نکته بسیار مهم! عملگر + یک جادوی دوگانه دارد.

- وقتی با اعداد (Integers/Floats) استفاده شود، کار جمع ریاضی را انجام می‌دهد؛
- وقتی با متن‌ها (Strings) استفاده شود، کار چسباندن^۸ را انجام می‌دهد.

این دو مثال را مقایسه کن:

```

1. # The magic of math (adding numbers)
2. num1 = 20
3. num2 = 5
4. result_math = num1 + num2
5. print("Result of math magic:")
6. print(result_math) # Output: 25
7.
8. # The magic of text (concatenating strings)
9. text1 = "20" # This is a string, not a number
10. text2 = "5" # This is also a string
11. result_text = text1 + text2
12. print("Result of text magic:")
13. print(result_text) # Output: "205"

```

این تفاوت بسیار مهم است و در فصل‌های آینده که می‌خواهیم ورودی بگیریم، دوباره با آن روبه‌رو خواهیم شد. فعلاً فقط یادت باشد: پایتون بین سکه ۲۰ و رشته "۲۰" تفاوت قائل می‌شود! (اولی عددست و دومی رشته‌ست و در زمان جمع رفتار متفاوتی دارند).

^۸ Concatenation

۴. پروژه: ساخت ماشین حساب جادویی - 100 xp

ما جراجو، وقت آن رسیده که طلسم‌های پراکنده‌ای که امروز یاد گرفتیم را با هم ترکیب کنیم (یعنی Numbers, Variables و Math Operators). می‌خواهیم اولین ابزار محاسباتی خودمان را بسازیم. در دنیای برنامه‌نویسی، همیشه ماشین حساب دم‌دست نداریم؛ ما خودمان ماشین حساب را می‌سازیم!

۴-۱. مأموریت: محاسبه غنائم

فرض کن ما دو صندوقچه گنج پیدا کرده‌ایم (دو عدد). هدف این است که برنامه‌ای بنویسیم که این دو عدد را بگیرد و بلافاصله تمام محاسبات حیاتی (جمع، تفریق، ضرب و تقسیم) را روی آنها انجام دهد و یک گزارش زیبا چاپ کند.

۴-۲. تصویر نهایی: این برنامه قرار است چه کار کند؟

قبل از کدنویسی، نتیجه را تصور کن. برنامه ما مثل یک لوح هوشمند عمل می‌کند. به محض اجرا شدن:

۱. به تو خوشامد می‌گوید؛
۲. دو عددی که در دل کد مخفی کرده‌ایم را به نمایش می‌گذارد؛
۳. و بعد ... بوم! نتایج جمع، تفریق، ضرب و تقسیم آنها را خیلی مرتب و زیر هم فهرست می‌کند. دیگر نیازی نیست چهار بار جداگانه حساب کنی، پایتون همه را یکجا انجام می‌دهد.

۴-۳. نقشه و طرح اولیه

بیا منطق این ابزار را مهندسی کنیم. ما سه مرحله اصلی داریم:

۱. **تعریف مواد اولیه (Variables):** ریاضیات بدون عدد معنا ندارد. ما باید دو جعبه (متغیر) بسازیم و عددهایمان را داخل آنها بریزیم.
 - **منطق:** ساخت متغیر num1 و num2 و مقداردهی به آنها (مثلاً ۱۵۰.۵ و ۲۵.۰).
۲. **انجام جادوی ریاضی (The Math Logic):** حالا باید موتور محاسبات را روشن کنیم.
 - **منطق:** چهار متغیر جدید می‌سازیم تا نتیجه جمع (+)، تفریق (-)، ضرب (*) و تقسیم (/) را در خودشان نگه دارند.
۳. **نمایش گزارش (The Display):** محاسبات که تمام شد، باید آنها را به کاربر نشان دهیم.
 - **منطق:** استفاده از دستور print برای چاپ کردن خطکشی‌های زیبا و نتایج نهایی.

۴-۴. چالش‌های فکری

قبل از اینکه کد را ببینی، سعی کن این معماها را حل کنی:

- **چالش ۱. نمادهای ممنوعه:** در ریاضیات روی کاغذ، برای ضرب از علامت $\times$ استفاده می‌کنیم. اما اگر این را در پایتون بنویسی، خطا می‌دهد! یادت هست نماد ضرب در دنیای کامپیوتر چه شکلی بود؟ (راهنمایی: $*$).
- **چالش ۲. ذخیره یا چاپ:** آیا بهتر است محاسبات را مستقیماً داخل پرانتز `print` انجام دهیم، یا اول آنها را در یک متغیر، مثلاً `sum_result`، ذخیره و بعد چاپ کنیم؟ (راهنمایی: ذخیره کردن در متغیر بهتر است، چون کد را تمیزتر و خواناتر می‌کند).

۴-۵. کد نهایی پروژه (The Spell)

حالا وقت نوشتن است. به فایل `project.py` برو و این کد را با دقت بنویس. سعی کن کامنت‌های انگلیسی را بخوانی تا بفهمی در هر خط چه اتفاقی می‌افتد.

```

1. # --- The Magic Calculator ---
2.
3. print("Welcome to the Magic Calculator!")
4. print("Today's numbers for calculation...")
5.
6. # --- Step 1: Define the loot (Variables) ---
7. # We define our numbers here. Try changing them later!
8. num1 = 150.5 # First treasure
9. num2 = 25.0  # Second treasure
10.
11. # Show the chosen numbers to the user
12. print("First number:")
13. print(num1)
14. print("Second number:")
15. print(num2)
16.
17. # --- Step 2: Perform Math Magic ---
18. # We store the results in new variables
19. sum_result = num1 + num2      # Addition
20. subtract_result = num1 - num2 # Subtraction
21. multiply_result = num1 * num2 # Multiplication
22. divide_result = num1 / num2  # Division
23.
24. # --- Step 3: Display the Results ---
25. print("=====")
26. print("Your magical calculation results:")
27.
28. print("Sum (+): ")
29. print(sum_result)
30.
31. print("Subtraction (-): ")
32. print(subtract_result)

```

```

33.
34. print("Multiplication (*): ")
35. print(multiply_result)
36.
37. print("Division (/): ")
38. print(divide_result)
39.
40. print("=====")

```

اجرا کن و نتیجه را ببین! خروجی تو باید دقیقاً مثل یک گزارش رسمی و زیبا باشد. بعد از اینکه اجرا کردی، برگرد و اعداد num1 و num2 را تغییر بده (مثلاً ۱۰۰ و ۱۰ بگذار) و دوباره اجرا کن. می‌بینی؟ ماشین حساب تو همیشه درست کار می‌کند!

۴-۶. جشن پیروزی و تحلیل

آفرین! شاید ساده به نظر برسد، اما تو الان اولین نرم‌افزار محاسباتی خودت را نوشتی. تو یاد گرفتی که:

۱. چطور داده‌ها را در متغیرها ذخیره کنی؛
 ۲. چطور از پایتون به عنوان یک ماشین حساب قدرتمند استفاده کنی؛
 ۳. چطور خروجی‌ها را مرتب و خوانا به کاربر نمایش دهی.
- این پایه و اساس تمام برنامه‌های مالی و مهندسی بزرگ دنیاست. ماجراجویی ادامه دارد...

| 🌟 ارتقاء لول: 🌟 |
۳ → ۵

مهارت‌های جدید:

◆ مفهوم Variable

له نام باستانی: جعبه‌های جادویی.

توضیح: قدرت نام‌گذاری بر اشیا. هر داده‌ای که نامی داشته باشد، در تسخیر توست.

◆ عملگرهای ریاضی

له نام باستانی: محاسبات کیهانی.

توضیح: جهان بر پایه اعداد بنا شده و تو اکنون معمار آنی.

فصل ۴: منطق و شرطها: ساخت بازی حدس عدد

مأموریت فصل: ساخت بازی حدس عدد و به چالش کشیدن کامپیوتر.

نقشهٔ راه:

- صحبت با ماجراجو: گرفتن ورودی از کاربر با `input()`؛
- تلهٔ کیمیاگری (تبدیل متن به عدد)؛
- دوراهی سرنوشت (`if` و `else`)؛
- مسیرهای چندگانه (`elif`)؛
- تکرار تا پیروزی (حلقهٔ `while`)؛
- کنترل قدرتمند حلقه: `break` و `continue`؛
- پروژه: پیاده‌سازی بازی حدس عدد.

۱. صحبت با ماجراجو: گرفتن ورودی از کاربر با `input()` جادویی

تا الان ما فقط با دستور `print` حرف می‌زدیم (اطلاعات می‌دادیم). حالا می‌خواهیم گوش کنیم (اطلاعات بگیریم). طلسم جادویی برای شنیدن، `input()` است. وقتی پایتون به این خط می‌رسد، توقف می‌کند و منتظر می‌ماند تا شما چیزی تایپ کنید و دکمه `Enter` را بزنید.

```
1. print("what is your name?")
2. name = input()
3.
4. print("hi " + name + "welcome")
```

نکته: طلسم `input` یک قابلیت باحال دارد. می‌توانی پیام را همان داخل پرانتزش بنویسی؛ این‌طوری کد کوتاه‌تر و تمیزتر می‌شود.

وقتی برنامه اجرا می‌شود، پایتون خودش آن پیام را نشان می‌دهد؛ دقیقاً انگار از `print` استفاده کردی. مثلاً:

```
1. name = input("what is your name?")
2. print("hi " + name + "welcome")
```

اینجا اول پیام «اسمت چیه؟» نمایش داده می‌شود، بعد چیزی که کاربر می‌نویسد داخل متغیر `name` ذخیره می‌شود و در آخر به او خوشامد می‌گوییم.

۲. تلهٔ کیمیاگری (تبدیل متن به عدد)

یک قانون خیلی مهم در پایتون وجود دارد: دستور `input` همیشه همه چیز را به صورت رشته^۱ می‌گیرد. حتی اگر شما عدد تایپ کنید!

اگر این کد را بنویسید، نتیجه فاجعه‌بار است:

```
1. # wrong code
2. num1 = input("num1:") # user enter -> 10
3. num2 = input("num2:") # user enter -> 20
4.
5. # python will "10" + "20" = "1020"
6. print(num1 + num2)
```

راه‌حل (کیمیاگری): باید متن را به عدد تبدیل کنیم تا پایتون به جای گذاشتن این دو عدد در کنار هم، آنها را با هم جمع کند. طلسم جادویی برای تبدیل کردن رشته به عدد صحیح، `int()` است.

^۱String


```
1. # fixed code
2. text1 = input("num1:") # user enter -> 10
3. num1 = int(text1) # python will cast "10" to 10
4.
5. text2 = input("num2:") # user enter -> 20
6. num2 = int(text2) # python will cast "20" to 20
7.
8. print(num1 + num2) # 10 + 20 = 30
```

۳. دوراهی سرنوشت (if و else)

اینجا مهم‌ترین بخش منطق برنامه‌نویسی است. می‌خواهیم بگوییم: «اگر رمز درست بود، در را باز کن، وگرنه آژیر بکش».

ما این کار را با **if** (یعنی اگر) و **else** (یعنی در غیر این صورت) انجام می‌دهیم.

۴. قانون طلایی تورفتگی^۲

پایتون چطور می‌فهمد کدام خط‌ها مربوط به باز کردن در و کدام خط‌ها مربوط به آژیر کشیدن است؟ جواب: با فاصله از سر خط.

هر وقت جلوی خطی علامت دونقطه : گذاشتید (مثل آخر خط if)، خط‌های بعدی باید کمی جلوتر شروع شوند (معمولاً به اندازه چهار فاصله یا یک دکمه Tab که بهتر است از Tab استفاده کنید).

این فاصله به پایتون می‌گوید که این خط‌ها به شرط بالایی متعلق‌اند.

بیایید در عمل ببینیم:

```
1. password = "123"
2. guess = input("Enter password: ")
3.
4. if guess == password:
5.     # We have an indentation (Tab) here.
6.     # This block runs ONLY if the condition is True
7.     print("Access Granted: Open the door")
8.
9. else:
10.    # We have an indentation (Tab) here too.
11.    # This block runs if the condition is False (Wrong password).
12.    print("Access Denied: Enable alarm")
13.
14. # This line has NO indentation.
15. # It is outside the if/else blocks and always runs.
```

Commented [GG9]: برای اولین بار در کد استفاده شده و در متن هیچ توضیحی در مورد آن داده نشده که اسم آن چیست و چه فرقی با = دارد

^۲Indentation

```
16. print("Continue program...")
```

تصور کن داری با استفاده از if و else دو تا جعبه می‌سازی که روی جعبه اول نوشته شده: «فقط در صورتی که این شرط رمز درست بود من را باز کن» و روی دیگری برعکس این نوشته شده است.

پایتون براساس آن شرط تو، در جعبه متناسب را باز و اجرایش می‌کند و از آن یکی جعبه کاملاً چشم‌پوشی کرده و رد می‌شود و دستورهای بعد از شرط را ادامه می‌دهد.

۴. مسیرهای چندگانه (elif)

اگر بخواهیم بیشتر از دو حالت داشته باشیم چه؟ مثلاً در بازی حدس عدد (بازی‌ای که در آن نفر اول یک عدد، مثلاً بین ۱ تا ۱۰ انتخاب می‌کند و نفر دوم آن را مثلاً ۵ حدس می‌زند و بعد نفر اول می‌گوید که عدد بالاتر، پایین‌تر یا درست است)، فقط درست و غلط نداریم، بلکه سه حالت داریم:

۱. عدد درست است (برنده).

۲. عدد شما کوچک است (راهنمایی کن).

۳. عدد شما بزرگ است (راهنمایی کن).

برای این کار از **elif** (مخفف Else If، یعنی در غیر این صورت اگر) استفاده می‌کنیم.

```
1. secret_number = 7
2. user_guess = int(input("Guess a number: "))
3.
4. if user_guess == secret_number:
5.     print("Great! You guessed correctly.")
6.
7. elif user_guess < secret_number:
8.     print("Your number is too small!")
9.
10. else:
11.     # If it is not equal and not smaller, it must be larger.
12.     print("Your number is too big!")
```

نکته: پایتون به ترتیب از بالا if را چک می‌کند. به محض اینکه یکی از شرط‌ها درست باشد، آن را اجرا کرده و بقیه elif و else را نادیده می‌گیرد.

۵. تکرار تا پیروزی (حلقه while)

مشکل کد بالا این است که فقط یک بار اجازه حدس زدن می‌دهد. ما می‌خواهیم بازی آن‌قدر تکرار شود تا بازیکن برنده شود.

طلسم جادویی **while** (یعنی تا زمانی که) دقیقاً همین کار را می‌کند. ساختار آن دقیقاً مثل **if** است (نیاز به دونقطه و تورفتگی دارد)، با این تفاوت که وقتی کدهای داخلش تمام شد، دوباره به بالا برمی‌گردد و شرط را چک می‌کند.

```
1. # Create a variable to control the game state
2. game_over = False
3.
4. while game_over == False:
5.     # The lines below repeat constantly because they are indented
6.     print("The game is running...")
7.
8.     answer = input("Do you want to end the game? (yes/no) ")
9.
10.    if answer == "yes":
11.        game_over = True # This makes the loop condition False, stopping the loop
12.
13.    print("Goodbye!")
```

نکته: وقتی داخل حلقه‌ایم، کدهای بعد از آن اجرا نمی‌شوند و حلقه دائماً ادامه پیدا می‌کند تا زمانی که این حلقه بالاخره تمام شود و سپس کدهای بعدیش شروع به اجرا شدن می‌کنند.

۶. کنترل قدرتمند حلقه: طلسم‌های **break** و **continue**

گاهی اوقات وسط اجرای یک طلسم تکرارشونده اتفاقی می‌افتد که می‌خواهیم فوراً حلقه را متوقف کنیم یا بخشی از آن را نادیده بگیریم. برای این کار، پایتون دو طلسم جادویی قدرتمند در اختیارمان می‌گذارد؛

۶-۱. دستور **break**: خروج اضطراری

تصور کنید در حال مبارزه در حلقه‌اید و ناگهان جان شما کم می‌شود. دیگر مهم نیست شرط حلقه چیست، شما باید همان لحظه فرار کنید! دستور **break** دقیقاً مثل شکستن شیشه اضطراری است؛ به محض اجرا شدن، حلقه بلافاصله نابود می‌شود و برنامه به خط بعد از حلقه می‌رود.

```
1. # Infinite loop seeking magic phrase
2. while True:
3.     text = input("Say the magic word: ")
4.
5.     if text == "Abra Kadabra":
6.         print("Spell broken! You are free.")
7.         break # The loop ends right here!
8.
9.     print("Wrong word! Try again.")
```

۶-۲. دستور continue: جاخالی دادن

گاهی نمی‌خواهید از حلقه خارج شوید، بلکه فقط می‌خواهید از روی یک مرحله خاص بپرید و به دور بعدی بروید. دستور continue به پایتون می‌گوید: «بی‌خیال بقیه کدهای این دور شو و سریع برو سراغ دور بعدی!».

مثلاً فرض کنید می‌خواهیم از  بشماریم، اما عدد ۳ را چاپ نکنیم:

```
1. number = 0
2. while number < 5:
3.     number = number + 1
4.
5.     if number == 3:
6.         print("Skipping the unlucky number!")
7.         continue # Jumps back to line 2 (start of loop)
8.
9.     print("Number is:", number)
```

Commented [GG10]: باتوجه به کد، باید ۵ باشد. درسته؟

خروجی: اعداد ۱، ۲، ۴ و ۵ چاپ می‌شوند (۳ چاپ نمی‌شود، چون قبل از print دستور continue اجرا شد).

نکته جادوگری: استفاده از break و while True یک الگوی بسیار رایج در بین برنامه‌نویسان حرفه‌ای است، چون کنترل کاملی به شما می‌دهد که دقیقاً کی و کجا بازی را تمام کنید.

۷. پروژه: بازی بزرگ حدس عدد^۳ - 100 xp

ماجراجو، تبریک می‌گوییم! تو ابزارهای قدرتمندی مثل input (گوش شنوا)، int (تبدیل‌گر)، if/else (تصمیم‌گیرنده) و while (حلقه زمان) را در اختیار داری.

حالا وقت آن است که تمام این طلسم‌ها را با هم ترکیب کنیم و یک بازی هوشمند بسازیم.

۷-۱. مأموریت: قفل رمزی دروازه

هدف ما ساختن یک نگهبان دیجیتالی است. ما یک عدد محرمانه (مثلاً ۱۴) را در دل برنامه مخفی می‌کنیم. کاربر باید حدس بزند این عدد چیست. اما نگهبان ما ساکت نمی‌ماند؛ او راهنمایی می‌کند که عدد کاربر بزرگ‌تر است یا کوچک‌تر تا بالاخره رمز درست پیدا شود.

^۳ Number Guessing Game

۲-۷. تصویر نهایی: این بازی چطور کار می‌کند؟

قبل از اینکه وارد کد شویم، بیا جریان بازی را تصور کنیم.

۱. برنامه اجرا می‌شود و می‌گوید: «من یک عدد بین ۱ تا ۲۰ انتخاب کرده‌ام»؛
۲. برنامه منتظر می‌ماند تا تو عددی را وارد کنی؛
۳. اگر گفتی ۵: برنامه می‌گوید: «خیلی کوچک است! بالاتر برو.» و دوباره منتظر می‌ماند؛
۴. اگر گفتی ۲۰: برنامه می‌گوید: «خیلی بزرگ است! پایین‌تر بیا.» و دوباره منتظر می‌ماند؛
۵. اگر گفتی ۱۴: برنامه می‌گوید: «آفرین! دروازه باز شد.» و بازی تمام می‌شود.

۳-۷. نقشه و طرح اولیه

بیا مثل یک مهندس نرم‌افزار، منطق پشت صحنه را طراحی کنیم. ما به ۳ بخش اصلی نیاز داریم:

۱. تنظیمات اولیه (The Setup):

- باید یک عدد را به عنوان رمز در یک متغیر ذخیره کنیم (مثلاً `secret_number`).
- به یک کلید نیاز داریم تا وضعیت بازی را کنترل کند. اول بازی، هنوز کسی برنده نشده، پس متغیر `correct_guess` را روی **False** (نادرست) می‌گذاریم.

۲. حلقه اصلی (The Loop):

- بازی نباید با یک حدس اشتباه تمام شود! ما می‌خواهیم تا زمانی که کاربر اشتباه می‌گوید، بازی ادامه پیدا کند.
- منطق: تا زمانی که **(while)** حدس درست نیست (`correct_guess == False`)، حلقه باید بچرخد.

۳. منطق مقایسه (The Logic):

- در هر دور از حلقه، باید عدد کاربر را بگیریم و با عدد رمز مقایسه کنیم.
- اینجا به ۳ شاخه نیاز داریم (**if**, **elif** و **else**):
 - آیا مساوی است؟ (برنده شدن و پایان حلقه).
 - آیا کوچک‌تر است؟ (راهنمایی: برو بالا).
 - آیا بزرگ‌تر است؟ (راهنمایی: بیا پایین).

۴-۷. چالش‌های فکری

قبل از دیدن کد، سعی کن این گره‌های ذهنی را باز کنی:

- **چالش ۱. طلسم تبدیل (The Type Conversion):** وقتی با دستور `input` عددی را از کاربر می‌گیریم، پایتون آن را به شکل متن می‌بیند (مثلاً "۱۴"). اما می‌خواهیم آن را با عدد ۱۴ مقایسه کنیم. با کدام دستور جادویی باید متن را به عدد صحیح تبدیل کنیم؟ (راهنمایی: `int()`).

Commented [GG11]: عبارت‌های انگلیسی هایلایت شده لازم به نظر نمی‌رسند. حذف شود؟

Commented [GG12]: بهتر نیست از رشته استفاده شود؟

- **چالش ۲. شکستن حلقه (Breaking the Loop):** حلقه `while` ما طوری تنظیم شده که تا وقتی `correct_guess` برابر با `False` است، تکرار شود. وقتی کاربر عدد درست را حدس زد، چه تغییری باید در متغیر `correct_guess` ایجاد کنیم تا حلقه متوقف شود؟ (راهنمایی: باید تبدیل به `True` شود).

- **چالش ۳. ترتیب شرط‌ها:** آیا ترتیب `if` و `elif` مهم است؟ خیلی مهم است؛ اگر هم‌زمان دو تا شرط `if` درست باشند، برنامه همیشه اولی را اجرا می‌کند و ترتیب مهم است. ولی در این مسئله چون شرط‌ها نمی‌توانند هم‌زمان درست باشند، ترتیب اهمیتی ندارد و نتیجه یکسان است).

Commented [GG14]: این هم راهنماییه؟

۵-۷. کد نهایی پروژه (The Spell)

حالا وقت نوشتن است. کد زیر را با دقت در فایل `project.py` بنویس.

تمرین ویژه: سعی کن یک بار از روی این کد بنویسی و بار دوم سعی کنی بدون نگاه کردن (یا فقط با نگاه کردن به نقشه بالا) آن را بازنویسی کنی.

```
1. # --- The Great Number Guessing Game ---
2.
3. # 1. Initial Setup
4. secret_number = 14      # The secret number creates the lock
5. correct_guess = False  # The gate is currently closed (False)
6.
7. print("Welcome to the Number Guessing Game!")
8. print("I have chosen a number between 1 and 20.")
9.
10. # 2. Start the Loop
11. # Repeat logic WHILE the guess is NOT correct yet
12. while correct_guess == False:
13.
14.     # Get input and VITAL STEP: convert text to integer
15.     guess_text = input("What is your guess? ")
16.     guess = int(guess_text)
```

*String

```

17.
18.     # 3. Check Game Logic (The Brain)
19.
20.     # Scene 1: The guess is correct
21.     if guess == secret_number:
22.         print("Excellent! You guessed correctly. The gate opens!")
23.         correct_guess = True # We flip the switch to 'True' to END the
loop
24.
25.     # Scene 2: The guess is too low
26.     elif guess < secret_number:
27.         print("Your number is too small! Try higher.")
28.
29.     # Scene 3: The guess is too high (Logic: if not equal and not
smaller, it must be bigger)
30.     else:
31.         print("Your number is too big! Try lower.")
32.
33. # 4. End of Game
34. # This line is outside the while loop (no indentation)
35. print("Game Over. Thanks for trying!")

```

اجراکن و جادو را ببین! برنامه را اجرا کن و سعی کن عدد مخفی را پیدا کنی.

بعداً می‌توانی secret_number را به هر عددی که دوست داری (مثلاً ۱۰۰۰) تغییر بدهی و دوستان
را به چالش بکشی!

۶-۷. جشن پیروزی و تحلیل

آفرین ماجراجو! الان یکی از مهم‌ترین مهارت‌های برنامه‌نویسی تاریخ را یاد گرفتی: تفکر منطقی.

تو یاد گرفتی چطور با if و else به کامپیوتر شعور ببخشی. تا قبل از این، کدهای ما فقط خط به خط پایین
می‌رفتند، اما الان کد تو تصمیم می‌گیرد که کدام خط را اجرا کند و کدام را نادیده بگیرد.

تمام هوش مصنوعی‌ها، بازی‌های کامپیوتری و ربات‌ها، از همین منطق ساده شروع می‌شوند. تو
حالا از یک کدنویس ساده به یک برنامه‌نویس متفکر تبدیل شدی.

ماجراجویی تازه شروع شده... ادامه بده!

Commented [GG15]: از نویسنده سوال شود: در طراحی برنامه

هیچ‌وقت به اجرای این خط نمی‌رسیم.. می‌تونیم حذفش کنیم یا
به تعدادی برای حدس‌ها بذاریم... چون به نظر این خط کارکردی
نداره. تنها وقتی بازیکن پیروز بازی می‌شود به او Game Over نشان
داده می‌شود

| 🌟 ارتقاء لول: 🌟 |

۱۰ → ۵

مهارت‌های جدید:

◆ ساختار Else If

له نام باستانی: دوراهی سرنوشت.

توضیح: دمیدن روح شعور در کالبد کد. برنامه اکنون می‌تواند بیاندیشد و انتخاب کند.

◆ حلقه While

له نام باستانی: طلسم پایداری.

توضیح: تکرار یک خواسته تا زمانی که حقیقت محقق شود.

◆ سطح جدید: F

توضیح: تبریک! به سطح جدیدی رسید.

۸. مأموریت‌های جانبی: تالار تمرین

ماجراجوی عزیز! قبل از اینکه وارد سرزمین‌های ناشناخته بخش دوم شوی، باید در تالار تمرین مهارت‌هایت را تقویت کنی. اینجا ۱۰ چالش پشت سر هم منتظر توست.

قانون تالار ساده است: تا وقتی حداقل یکبار خودت تلاش نکردی، به پاسخ‌نامه نگاه نکن!

۸-۱. مأموریت‌ها

۱. مأموریت: نشان ماجراجو^۵ - 20 xp

اولین قدم برای ورود به انجمن، معرفی خودت است.

- **وظیفه:** برنامه‌ای بنویس که فقط یک خط کد داشته باشد و جمله "I am a Python Adventurer!" را روی صفحه چاپ کند.
- **هدف:** مطمئن شویم محیط کدنویسی تو درست کار می‌کند.

۲. مأموریت: دره پژواک^۶ - 20 xp

در این دره جادویی، هر کلمه‌ای بگویی چند بار تکرار می‌شود.

- **وظیفه:** یک کلمه از کاربر بگیر. سپس یک عدد از کاربر بگیر. برنامه باید آن کلمه را به تعداد آن عدد تکرار و چاپ کند.
- **هدف:** یادگیری ضرب کردن رشته‌ها در پایتون (مثلاً `"Hello" * 3`).

۳. مأموریت: تبدیل گر طلا^۷ - 20 xp

در بازار شهر، هر ۱ سکه طلا معادل ۱۰ سکه نقره است.

- **وظیفه:** برنامه‌ای بنویس که تعداد سکه‌های طلا را از کاربر بگیرد (ورودی)، آن را در ۱۰ ضرب کند و تعداد سکه‌های نقره را نشان دهد.
- **هدف:** تبدیل ورودی متنی به عدد و انجام محاسبات ریاضی.

^۵ The ID Badge

^۶ Echo Canyon

^۷ The Gold Converter

۴. مأموریت: نفس اژدها^۸ – 20 xp

اژدهای نگهبان فقط دمای فارنهایت را می‌فهمد، اما دماسنج ما به سانتی‌گراد است.

- **وظیفه:** دمای سانتی‌گراد را بگیر و به فارنهایت تبدیل کن. (فرمول: ورودی ضربدر ۱.۸، بعلاوه^۹ ۳۲).

- **هدف:** کار با اعداد اعشاری و فرمول‌نویسی.

۵. مأموریت: نگهبان دروازه^۹ – 20 xp

به یک در بسته رسیده‌ای که فقط با رمز عبور باز می‌شود. رمز صحیح عدد 1234 است.

- **وظیفه:** یک عدد به عنوان رمز از کاربر بگیر. اگر رمز برابر با 1234 بود، "Welcome!" را چاپ کن و در غیر این صورت (else)، چاپ کن "Access Denied!".
- **هدف:** درک شرط ساده if/else.

۶. مأموریت: چراغ راهنما^{۱۰} – 20 xp

ربات‌های شهر باید قوانین راهنمایی را بلد باشند.

- **وظیفه:** رنگ چراغ را از کاربر بپرس (red, yellow یا green).
 - اگر red بود: چاپ کن "Stop".
 - اگر yellow بود: چاپ کن "Get Ready".
 - اگر green بود: چاپ کن "Go".
 - برای هرچیز دیگر: چاپ کن "Unknown Color".
- **هدف:** مدیریت چند شرط با elif.

Commented [GG16]: تکلیف عددها روشن نیست. فارسی یا انگلیسی؟

^۸ The Dragon's Breath

^۹ The Gatekeeper

^{۱۰} Traffic Light

۷. مأموریت: معمار اهرام^{۱۱} - 20 xp

برای رسیدن به قلعه آسمانی، باید پله‌هایی از جنس نور بسازی.

- **وظیفه:** یک عدد از کاربر بگیر (مثلاً ۵). برنامه‌ای بنویس که با استفاده از حلقه، به تعداد آن عدد، پله بسازد و چاپ کند.
 - پله اول: *
 - پله دوم: **
 - پله سوم: ***
 - و ...
- **هدف:** استفاده از حلقه while و جادوی ضرب رشته‌ها (تکرار کاراکتر).

۸. مأموریت: پرتاب موشک^{۱۲} - 20 xp

وقت هیجان است! می‌خواهیم یک موشک به فضا بفرستیم.

- **وظیفه:** یک شمارش معکوس بساز. یک متغیر با مقدار ۱۰^{۱۳} بساز. با استفاده از حلقه (while)، تا زمانی که عدد بزرگ‌تر از ۰ است، آن را چاپ کن و یکی از آن کم کن. وقتی حلقه تمام شد، چاپ کن "Liftoff!".
- **هدف:** درک ساختار حلقه کاهش.

۹. مأموریت: صندوق داری نهایت^{۱۳} - 20 xp

شما مسئول صندوق فروشگاه‌اید. مشتری‌ها تعداد نامشخصی کالا می‌خرند.

- **وظیفه:** یک برنامه بنویس که در یک حلقه همیشگی (True)، قیمت کالا را از کاربر بپرسد.
 - قیمت‌ها را با هم جمع کن (یک متغیر برای مجموع بساز).
 - اگر کاربر عدد ۰ را وارد کرد، حلقه را بشکن (break) و جمع کل فاکتور را نمایش بده.

Commented [GG17]: عددها بعضی جاها فارسی و جاهایی انگلیسیه. به نظرم بهتره یکدست بشه

¹¹ The Pyramid Builder

¹² Rocket Launch

¹³ The Infinite Cashier

- هدف: استفاده از break و متغیر جمع‌کننده^{۱۴}.

۱۰. مأموریت: قهوه‌ساز هوشمند^{۱۵} - 20 xp

و اما چالش نهایی! یک دستگاه قهوه‌ساز بساز که هم قیمت را چک کند و هم بقیه پول^{۱۶} را پس بدهد.

- سناریو: قیمت یک قهوه ۳/۵۰ دلار است.

- وظیفه:

۱. مبلغ پول را از کاربر بگیر (مثلاً ۵ دلار).

۲. اگر پول کمتر از ۳/۵۰ دلار بود، بنویس "Not enough money".

۳. اگر پول کافی بود، قهوه را بده و بقیه پول را حساب کن و به کاربر نمایش بده.

- هدف: یک پروژه کامل با ورودی، تبدیل نوع، شرط و ریاضی.

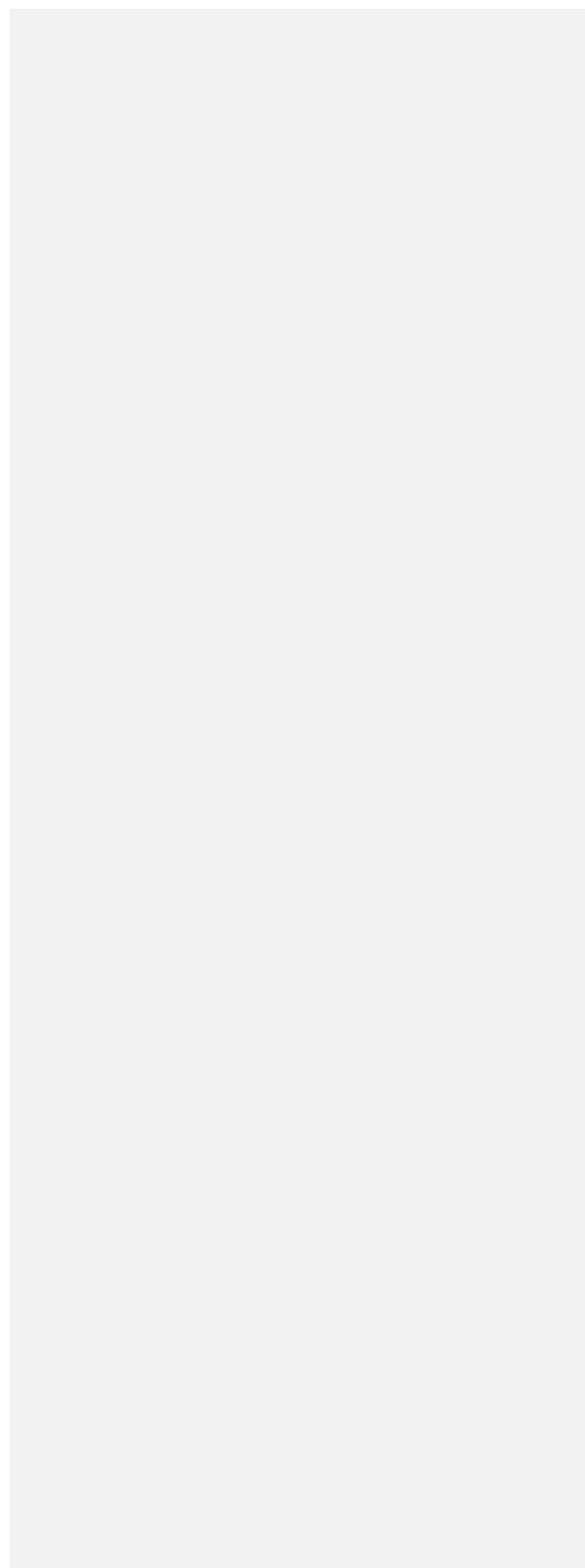
آیا سفرت را در این مرحله کامل کرده‌ای، ماجراجو؟

اگر بله، کتاب را ورق بزن؛ کتابخانه سایه‌ها (نتیجه مأموریت‌ها) در صفحه بعد منتظر توست.

¹⁴ Accumulator

¹⁵ The Smart Coffee Machine

¹⁶ Change



۸-۲. کتابخانه سایه‌ها (نتیجه مأموریت‌ها)

۱. مأموریت: نشان ماجراجو

```
1. # Just a simple print statement
2. print("I am a Python Adventurer!")
```

۲. مأموریت: دره پژواک

نکته: در پایتون می‌توانید یک رشته را در یک عدد ضرب کنید تا تکرار شود.

```
1. word = input("Say something: ")
2. count = int(input("How many times? "))
3.
4. # Repeating the string
5. print(word * count)
```

۳. مأموریت: تبدیل گرطلا

نکته: ورودی‌های input همیشه به صورت متن (متغیر رشته‌ای) هستند، پس برای انجام محاسبات ریاضی باید با int() آنها را به عدد تبدیل کنید.

```
1. gold = int(input("Enter gold coins: "))
2. silver = gold * 10
3. print(f"You have {silver} silver coins.")
```

۴. مأموریت: نفس ازدها

نکته: برای دما و پول معمولاً از اعداد اعشاری استفاده می‌کنیم.

```
1. celsius = float(input("Enter Temperature in C: "))
2. fahrenheit = (celsius * 1.8) + 32
3.
4. print(f"Temperature in F: {fahrenheit}")
```

۵. مأموریت: نگهبان دروازه

نکته: در برنامه‌نویسی برای مقایسه کردن دو مقدار از دو علامت مساوی == استفاده می‌کنیم.

```
1. password = int(input("Enter password: "))
2.
3. if password == 1234:
4.     print("Welcome!")
5. else:
6.     print("Access Denied!")
```

۶. مأموریت: چراغ راهنما

نکته: زمانی که بیش از دو حالت شرطی داریم (مثل قرمز، زرد و سبز)، از دستور elif استفاده می‌کنیم.

```
1. color = input("Enter light color (red/yellow/green): ")
2.
3. if color == "red":
4.     print("Stop")
5. elif color == "yellow":
6.     print("Get Ready")
7. elif color == "green":
8.     print("Go")
9. else:
10.    print("Unknown Color")
```

۷. مأموریت: معمار اهرام

نکته: در پایتون اگر یک رشته (مثل *) را در یک عدد ضرب کنید، به همان تعداد تکرار می‌شود.

```
1. height = int(input("Enter height: "))
2. step = 1
3.
4. while step <= height:
5.     print("*" * step)
6.     step = step + 1
```

۸. مأموریت: پرتاب موشک

نکته: فراموش نکنید که داخل حلقه، حتماً از متغیر شمارنده کم کنید ($n = n - 1$)، وگرنه حلقه تا ابد ادامه می‌یابد!

```
1. n = 10
2.
3. while n > 0:
4.     print(n)
5.     n = n - 1
6.
7. print("Liftoff!")
```

۹. مأموریت: صندوق داری نهایت

نکته: ما یک متغیر به نام total بیرون از حلقه نیاز داریم تا مجموع قیمت‌ها را در آن ذخیره کنیم.

```
1. print("--- Enter prices (Type 0 to finish) ---")
2. total = 0
3.
4. while True:
5.     price = float(input("Price: $"))
6.
7.     # If price is 0, stop the loop
8.     if price == 0:
9.         break
10.
11.     # Add price to total
12.     total = total + price
13.
14. print(f"Total Bill: ${total}")
```

۱۰. مأموریت: قهوه‌ساز هوشمند

نکته: این پروژه ترکیبی از ریاضی (تفریق هزینه از پول نقد) و شرط (بررسی کافی بودن پول) است.

```
1. print("--- Coffee Price: $3.50 ---")
2. cash = float(input("Insert money: $"))
3.
4. if cash < 3.50:
5.     print("Not enough money. Money returned.")
6. else:
7.     change = cash - 3.50
8.     print("Here is your coffee")
9.     print(f"Your change is: ${change}")
```